

Hands-On Report

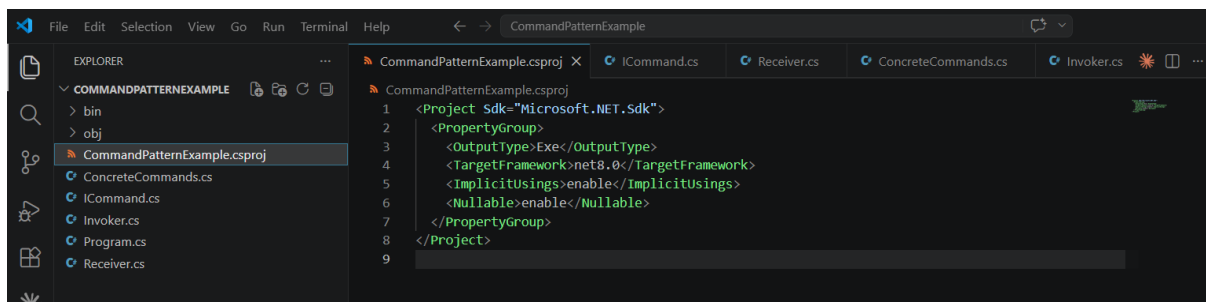
Exercise 9: Implementing the Command Pattern

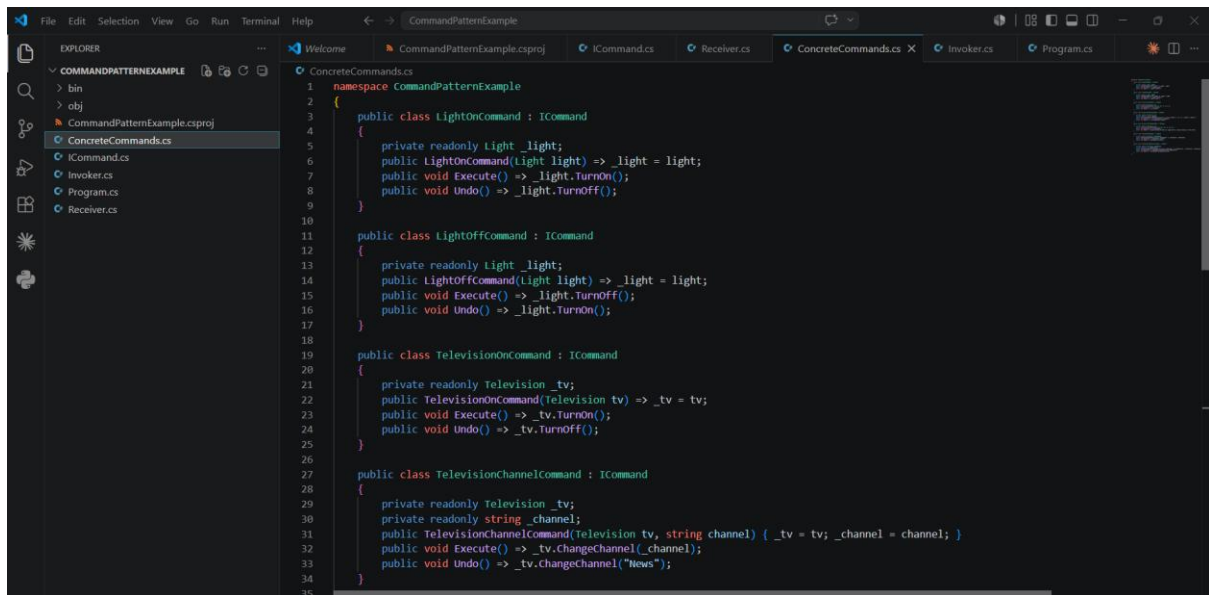
In this exercise, I implemented and tested the Command Pattern in a .NET application by creating a project named

CommandPatternExample. I designed a **Command** interface with an **Execute()** method and implemented command classes such as **LightOnCommand** and **LightOffCommand** to perform different operations. I also developed a **Light** class as the receiver and a **RemoteControl** class as the invoker to execute the assigned commands.

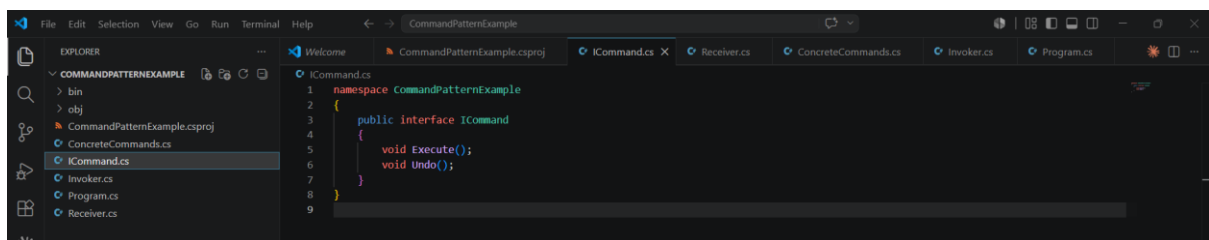
After executing the application, I tested different commands through the remote control to ensure that the light responded correctly to each action. This practical exercise helped me understand how the Command Pattern encapsulates requests as objects, separates the sender from the receiver, and supports features such as request queuing, logging, and undo operations, resulting in more flexible and maintainable .NET applications.

Coding:

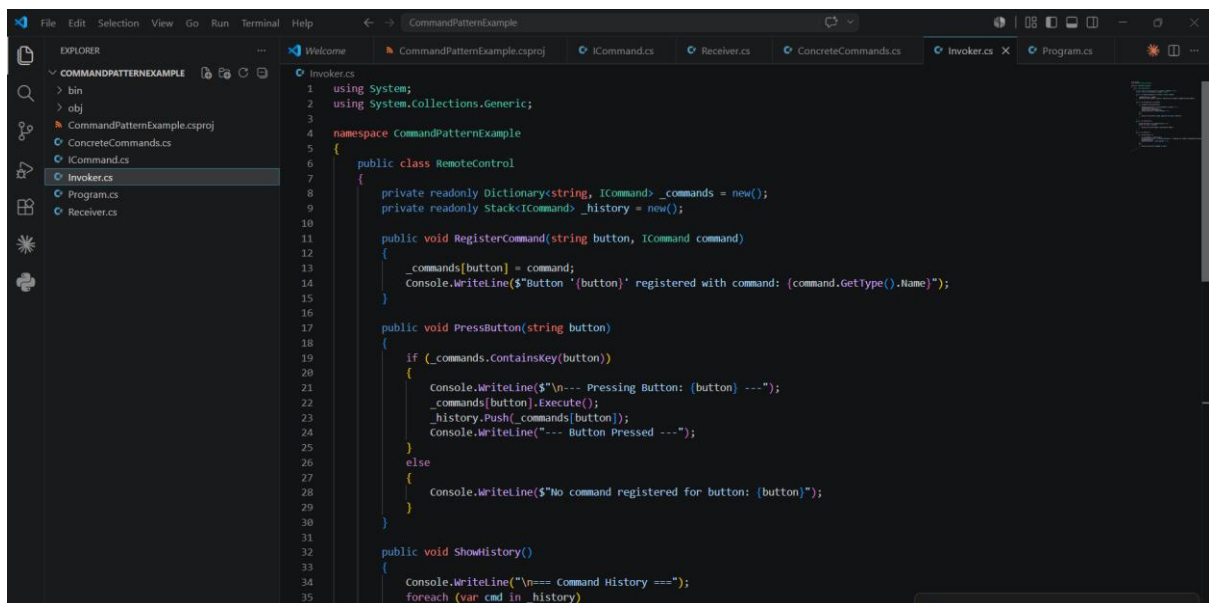




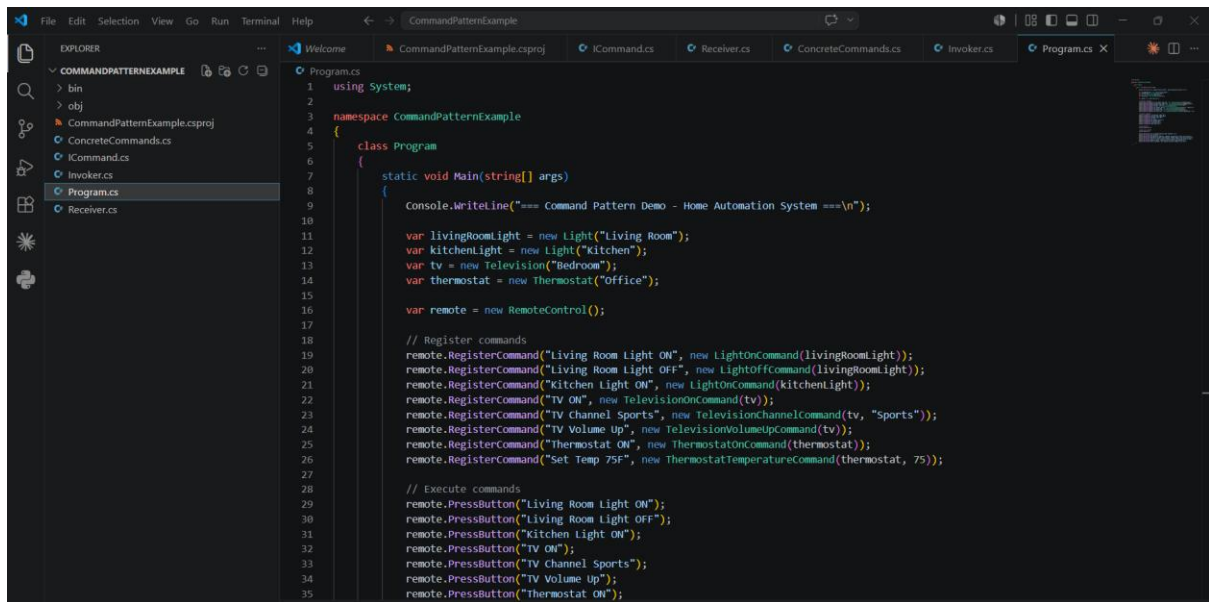
```
1 namespace CommandPatternExample
2 {
3     public class LightOnCommand : ICommand
4     {
5         private readonly Light _light;
6         public LightOnCommand(Light light) => _light = light;
7         public void Execute() => _light.TurnOn();
8         public void Undo() => _light.TurnOff();
9     }
10
11     public class LightOffCommand : ICommand
12     {
13         private readonly Light _light;
14         public LightOffCommand(Light light) => _light = light;
15         public void Execute() => _light.TurnOff();
16         public void Undo() => _light.TurnOn();
17     }
18
19     public class TelevisionOnCommand : ICommand
20     {
21         private readonly Television _tv;
22         public TelevisionOnCommand(Television tv) => _tv = tv;
23         public void Execute() => _tv.TurnOn();
24         public void Undo() => _tv.TurnOff();
25     }
26
27     public class TelevisionChannelCommand : ICommand
28     {
29         private readonly Television _tv;
30         private readonly string _channel;
31         public TelevisionChannelCommand(Television tv, string channel) { _tv = tv; _channel = channel; }
32         public void Execute() => _tv.ChangeChannel(_channel);
33         public void Undo() => _tv.ChangeChannel("News");
34     }
35 }
```



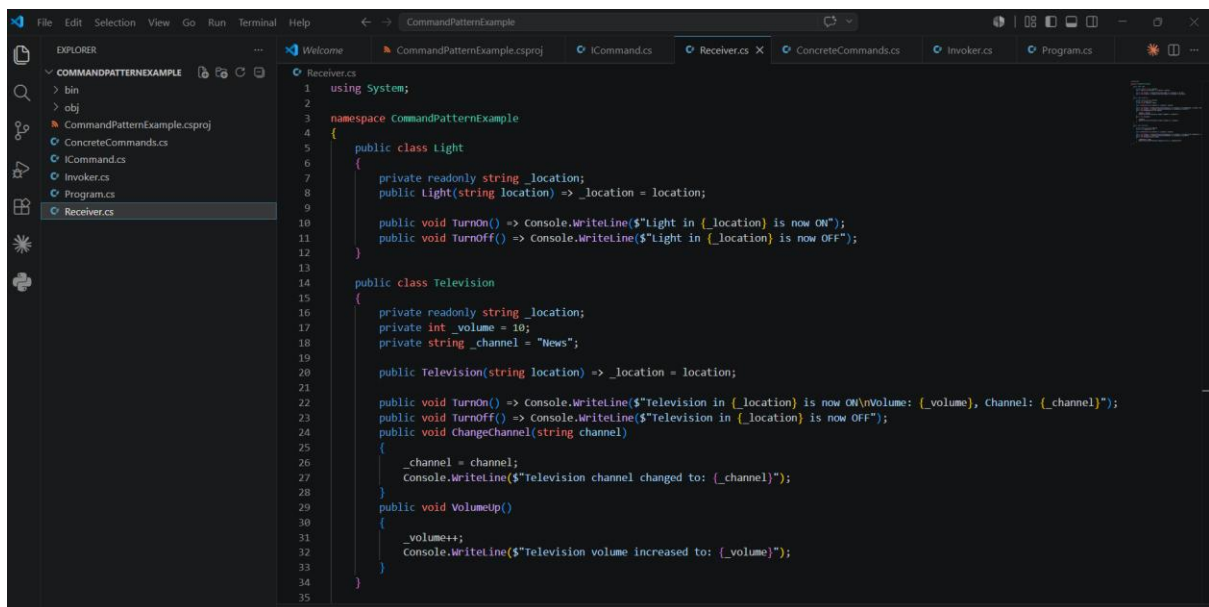
```
1 namespace CommandPatternExample
2 {
3     public interface ICommand
4     {
5         void Execute();
6         void Undo();
7     }
8 }
9
```



```
1 using System;
2 using System.Collections.Generic;
3
4 namespace CommandPatternExample
5 {
6     public class RemoteControl
7     {
8         private readonly Dictionary<string, ICommand> _commands = new();
9         private readonly Stack<ICommand> _history = new();
10
11         public void RegisterCommand(string button, ICommand command)
12         {
13             _commands[button] = command;
14             Console.WriteLine($"Button '{button}' registered with command: {command.GetType().Name}");
15         }
16
17         public void PressButton(string button)
18         {
19             if (_commands.ContainsKey(button))
20             {
21                 Console.WriteLine($"\\n--- Pressing Button: {button} ---");
22                 _commands[button].Execute();
23                 _history.Push(_commands[button]);
24                 Console.WriteLine("--- Button Pressed ---");
25             }
26             else
27             {
28                 Console.WriteLine($"No command registered for button: {button}");
29             }
30         }
31
32         public void ShowHistory()
33         {
34             Console.WriteLine("\\n== Command History ==");
35             foreach (var cmd in _history)
```



```
1 using System;
2
3 namespace CommandPatternExample
4 {
5     class Program
6     {
7         static void Main(string[] args)
8         {
9             Console.WriteLine("=== Command Pattern Demo - Home Automation System ===\n");
10
11             var livingRoomLight = new Light("Living Room");
12             var kitchenLight = new Light("Kitchen");
13             var tv = new Television("Bedroom");
14             var thermostat = new Thermostat("Office");
15
16             var remote = new RemoteControl();
17
18             // Register commands
19             remote.RegisterCommand("Living Room Light ON", new LightOnCommand(livingRoomLight));
20             remote.RegisterCommand("Living Room Light OFF", new LightOffCommand(livingRoomLight));
21             remote.RegisterCommand("Kitchen Light ON", new LightOnCommand(kitchenLight));
22             remote.RegisterCommand("TV ON", new TelevisionOnCommand(tv));
23             remote.RegisterCommand("TV channel Sports", new TelevisionChannelCommand(tv, "Sports"));
24             remote.RegisterCommand("TV Volume Up", new TelevisionVolumeUpCommand(tv));
25             remote.RegisterCommand("Thermostat ON", new ThermostatOnCommand(thermostat));
26             remote.RegisterCommand("Set Temp 75F", new ThermostatTemperatureCommand(thermostat, 75));
27
28             // Execute commands
29             remote.PressButton("Living Room Light ON");
30             remote.PressButton("Living Room Light OFF");
31             remote.PressButton("Kitchen Light ON");
32             remote.PressButton("TV ON");
33             remote.PressButton("TV channel Sports");
34             remote.PressButton("TV Volume Up");
35             remote.PressButton("Thermostat ON");
36         }
37     }
38 }
```



```
1 using System;
2
3 namespace CommandPatternExample
4 {
5     public class Light
6     {
7         private readonly string _location;
8         public Light(string location) => _location = location;
9
10        public void TurnOn() => Console.WriteLine($"Light in {_location} is now ON");
11        public void TurnOff() => Console.WriteLine($"Light in {_location} is now OFF");
12    }
13
14    public class Television
15    {
16        private readonly string _location;
17        private int _volume = 10;
18        private string _channel = "News";
19
20        public Television(string location) => _location = location;
21
22        public void TurnOn() => Console.WriteLine($"Television in {_location} is now ON\nVolume: {_volume}, channel: {_channel}");
23        public void TurnOff() => Console.WriteLine($"Television in {_location} is now OFF");
24        public void ChangeChannel(string channel)
25        {
26            _channel = channel;
27            Console.WriteLine($"Television channel changed to: {_channel}");
28        }
29        public void VolumeUp()
30        {
31            _volume++;
32            Console.WriteLine($"Television volume increased to: {_volume}");
33        }
34    }
35 }
```

